

ModularML: A Backend-Agnostic, Declarative Framework for Reproducible and Modular Machine Learning Experiments

Ben Nowacki¹, Tingkai Li¹, Mohammad Mundiwala¹, Hui Hua¹, Fei Miao¹, and Chao Hu¹

¹ University of Connecticut, United States ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Open Journals](#)

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Modern machine learning (ML), particularly deep learning, has become central to scientific research across domains such as energy systems, materials science, structural health monitoring, and biomedical engineering. However, most ML frameworks remain optimized for users with formal computer science backgrounds. As a result, domain researchers often rely on fragmented scripts, backend-specific conventions, and ad hoc experiment management workflows that limit reproducibility and collaboration.

ModularML is an open-source Python framework designed to address this gap. It provides a backend-agnostic, declarative, and fully serializable architecture for ML experimentation, enabling users to define complete workflows—including data curation, feature processing, model architecture, training strategy, loss configuration, and evaluation—using modular, composable components that serialize into a single reproducible artifact.

At its core, ModularML introduces: * a FeatureSet abstraction for structured, backend-agnostic data handling and splitting, * a directed acyclic ModelGraph composed of independent ModelNodes, * multi-phase training orchestration with configurable freeze/unfreeze policies and phase- and node-specific loss definitions, * and full pipeline serialization and visualization for transparent sharing and inspection of experiments.

By separating experiment definition from backend implementation, ModularML allows researchers to collaborate and audit ML studies without requiring expertise in PyTorch (?), Keras (Chollet & others, 2015), or other specific ML libraries.

Statement of need

ML workflows in academia differ substantially from typical industry-focused ML development. Scientific researchers emphasize experimentation with domain-specific feature selection, sampling, and model architecture. Existing libraries such as PyTorch Lightning (W. Falcon & The PyTorch Lightning team, 2019), Catalyst (Kolesnikov, 2018), FastAI (Howard & others, 2018), and PyTorch Ignite (Fomin et al., 2020) can streamline training loops, but typically remain tied to a single backend, prioritize development productivity over experimentation standardization, and depend on implicit code-based configurations with limited traceability of dataset splitting.

In contrast, ModularML is built around the principle of *configuration-as-contract*: the entire ML experiment is represented as a structured, inspectable object. This design offers three major benefits:

- 39 1. **Reproducibility through serialization.** Entire experiments, from initial data curation
40 to multi-stage training and evaluation, can be saved and shared as a single file. A
41 collaborator can load this artifact and inspect the full pipeline without having to trace
42 the source code.
 - 43 2. **Backend abstraction.** Model nodes are agnostic to the backend in which the underlying
44 model is defined. ModularML does not replace existing ML libraries such as PyTorch,
45 Keras, or scikit-learn (Pedregosa et al., 2011); rather, it improves their usage by
46 providing a unified interface that researchers can intuitively understand. This backend-
47 agnostic design also enables mixed-backend experiments, allowing direct reuse of a
48 collaborator's model regardless of which library it uses.
 - 49 3. **Transparency for non-ML experts.** Built-in summary and visualize methods for feature
50 sets, model graphs, and experiment execution phases (`modularml.ExperimentPhase`)
51 allow users and code reviewers to quickly inspect data splits, model architecture,
52 loss routing, and training sequencing without needing to understand backend-specific
53 implementation details.
- 54 These features directly address common reproducibility and collaboration challenges in research,
55 while still supporting a comprehensive suite of modeling techniques and existing ML libraries.

56 State of the field

57 General-purpose deep learning frameworks (e.g., PyTorch, Keras) streamline model training
58 but typically assume homogeneous backends and single-phase workflows. Data-centric libraries
59 (e.g., Apache Arrow ("Arrow," n.d.), Hugging Face (Lhoest et al., 2021)) manage datasets
60 but leave sampling and experiment scheduling to custom user code. Experiment managers
61 (e.g., MLflow (Zaharia et al., 2018), Weights & Biases ("Wandb," n.d.)) track runs but do
62 not define how data flows through modular graphs.

63 ModularML sits between these layers. It provides concrete abstractions for data
64 (`modularml.FeatureSet`), sampling logic (`modularml.Sampler`), modeling (`modularml.ModelGraph`
65 composed of `modularml.ModelNode`), losses (`modularml.AppliedLoss`), and orchestration
66 (`modularml.ExperimentPhase`), while remaining backend-agnostic and lightweight enough for
67 research scripts or notebooks.

68 Software design

69 The ModularML architecture provides abstractions for data storage and processing, graph-based
70 modeling, and execution of distinct training sequences (Figure 1).

ModularML architecture overview: FeatureSet curation into samples with feature, target, and tag domains; flexible ModelGraph construction for rapid experimentation of model topology; and multi-phase training workflows sequenced within a single Experiment container.

Figure 1: ModularML architecture overview: FeatureSet curation into samples with feature, target, and tag domains; flexible ModelGraph construction for rapid experimentation of model topology; and multi-phase training workflows sequenced within a single Experiment container.

71 FeatureSets

72 A `modularml.FeatureSet` organizes data into three intent-driven domains—features, targets,
73 and tags—reflecting the core design philosophy of ModularML. Features represent model inputs
74 (what the model learns from), targets represent model outputs (what the model is trained to
75 predict or reproduce), and tags store optional metadata associated with each sample.

Each sample, defined by its feature, target, and tag attributes, is assigned a globally unique identifier to ensure explicit traceability throughout the entire lifecycle of an experiment. This identifier propagates through splitting, sampling, batching, model execution, and evaluation, enabling transparent lineage tracking and reproducible analysis.

Data within a `modularml.FeatureSet` is stored in backend-agnostic containers supporting NumPy (Harris et al., 2020) arrays, Pandas (team, 2020) dataframes, PyTorch tensors, and TensorFlow (Abadi et al., 2015) tensors. All downstream operations, such as splitting, subsetting, and batching, operate using no-copy views of the underlying data. This design makes split definitions explicit and inspectable, reduces memory overhead, and minimizes the risk of data leakage or unintended experimentation bias.

Sampling

All subclasses of `modularml.Sampler` consume a `modularml.FeatureSet`, or subset views of one, and emit aligned batches, supporting stratification, grouping, and multi-role sampling needed for contrastive or paired training schemes.

ModelGraph

The full experiment model is defined as a directed acyclic graph (DAG) of `modularml.ModelNodes`, where each node wraps a user-defined or built-in model and exposes optimizer hooks, build routines, and freeze/unfreeze controls. Nodes are subset into several subclasses (e.g., `modularml.MergeNode`) depending on the connectivity constraints of each. A `modularml.ModelNode` supports single-input, single-output operation, whereas the `modularml.MergeNode` supports multiple inputs. `modularml.MergeNode` has expensive support for merge logic, including concatenation axes, aggregation operators, and padding, providing experimentation with branding and multi-task ML models. All nodes belong to an overarching `modularml.ModelGraph` which handles DAG-ordered execution during both forward and backward propagation with either per-node or global optimizers.

Losses and phases

`modularml.AppliedLoss` binds objectives to nodes, enabling composite-loss training and targeted loss aggregation over specific nodes and loss types. These node-targeted losses are recorded for all execution phases, providing tracked losses with fast filtering and analysis upon execution completion.

`TrainPhase` orchestrates iterative training with callbacks and checkpointing, `FitPhase` performs single-pass fit-based workflows (e.g., for scikit-learn estimators), and `EvalPhase` runs inference-only sweeps.

Experiment orchestration

The `modularml.Experiment` class serves as the top-level container for a machine learning workflow, binding together one or more `modularml.FeatureSets`, a `modularml.ModelGraph`, and a sequence of execution phases. Each `modularml.ExperimentPhase` defines a reproducible unit of execution, including data sampling, loss definitions, callbacks, and training or evaluation behavior.

By explicitly separating experiment definition from execution, ModularML supports multi-stage workflows such as pretraining, fine-tuning, and evaluation while retaining all information required to reproduce each phase. The `modularml.Experiment` also provides serialization, checkpointing, and execution tracking, allowing complete experiments to be exported, shared, and reloaded as self-contained artifacts.

Research impact statement

ModularML lowers the barrier for scientists prototyping hybrid ML systems that combine learned encoders, classical regressors/classifiers, and domain-specific samplers. By guaranteeing that every component can be serialized, checkpointed, and replayed, the framework supports reproducible experiments and facilitates sharing of trained graphs or datasets between collaborators. Its backend-agnostic graph execution simplifies comparisons across PyTorch, TensorFlow, and scikit-learn implementations, encouraging rigorous benchmarking and cross-validation in applied research domains.

The unique contributions can be summarized as follows:

1. **Full pipeline serialization.** Rather than serializing only model weights, ModularML serializes data definitions with assigned unique identifiers, split logic, sampling configuration, graph topology, per-phase loss routing, and training and evaluation configurations. This enables complete auditability of published ML studies, as shown in [Figure 2](#).
2. **Backend-agnostic DAG modeling.** By supporting mixed-backend modeling, ModularML extends adoption of existing ML libraries rather than replacing them, reducing friction between works that use different packages.
3. **Declarative syntax.** All aspects of an ML experiment are constructed via declarative, configuration-driven objects. This reduces boilerplate, simplifies code review, and significantly accelerates experimentation across different training techniques and model architectures.
4. **Built-in visualization for validation.** Visual summaries help detect data leakage, misconfigured sampling, incorrect loss routing, unexpected model topologies, and training sequencing issues ([Figure 2](#)). This enables validation at definition time, ensures execution matches intent, and lowers the barrier for domain scientists to review ML-based results.

Backend-agnostic and fully serializable experiment workflow in ModularML with built-in visualization utilities.

Figure 2: Backend-agnostic and fully serializable experiment workflow in ModularML with built-in visualization utilities.

Mathematics

ModularML does not introduce new mathematical formulations; instead, it codifies well-established training loops, loss aggregation, and optimizer steps across common ML backends. Any model- or loss-specific mathematics is delegated to user-defined modules or external libraries, ensuring that ModularML remains an orchestration layer that provides continued support for existing ML packages.

AI usage disclosure

Generative AI tools were used in a limited capacity during the development of this software, confined to helping structure docstrings and scaffold unit tests. All AI-assisted contributions were reviewed and verified by the authors. AI was not used in the writing of this manuscript.

Acknowledgements

We thank the ModularML contributor community for feature ideas, bug reports, and documentation improvements, and acknowledge Professor Ryan Cooper for guidance in navigating the open-source environment.

References

- Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., ... Zheng, X. (2015). *TensorFlow: Large-scale machine learning on heterogeneous systems*. <https://www.tensorflow.org/>
- Arrow. (n.d.). In *GitHub repository*. GitHub. <https://github.com/apache/arrow>
- Chollet, F., & others. (2015). *Keras*. <https://keras.io>
- Fomin, V., Anmol, J., Desroziers, S., Kriss, J., & Tejani, A. (2020). High-level library to help with training neural networks in PyTorch. In *GitHub repository*. GitHub. <https://github.com/pytorch/ignite>
- Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk, M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Howard, J., & others. (2018). *Fastai*. GitHub. <https://github.com/fastai/fastai>
- Kolesnikov, S. (2018). Catalyst - accelerated deep learning r&d. In *GitHub repository*. GitHub. <https://github.com/catalyst-team/catalyst>
- Lhoest, Q., Villanova del Moral, A., Platen, P. von, Wolf, T., Šaško, M., Jernite, Y., Thakur, A., Tunstall, L., Patil, S., Drame, M., Chaumond, J., Plu, J., Davison, J., Brandeis, S., Sanh, V., Le Scao, T., Canwen Xu, K., Patry, N., Liu, S., ... Delangue, C. (2021). Datasets: A Community Library for Natural Language Processing. *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 175–184. <https://aclanthology.org/2021.emnlp-demo.21>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- team, T. pandas development. (2020). *Pandas-dev/pandas: pandas (latest)*. Zenodo. <https://doi.org/10.5281/zenodo.3509134>
- W. Falcon, & The PyTorch Lightning team. (2019). PyTorch lightning (version 1.4). In *GitHub Repository*. GitHub. <https://doi.org/10.5281/zenodo.3828935>
- Wandb. (n.d.). In *GitHub repository*. GitHub. <https://github.com/wandb/wandb>
- Zaharia, M. A., Chen, A., Davidson, A., Ghodsi, A., Hong, S. A., Konwinski, A., Murching, S., Nykodym, T., Ogilvie, P., Parkhe, M., Xie, F., & Zumar, C. (2018). Accelerating the Machine Learning Lifecycle with MLflow. *IEEE Data Eng. Bull.*, 41, 39–45. <https://api.semanticscholar.org/CorpusID:83459546>